RIPHAH INTERNATIONAL UNIVERSITY
Faculty of Computing
Department of Computer Science
Islamabad, I-14

Online Bidding Management System

Database Management System — Semester Project

Name:	Muhammad Ubaid ur Rehman
Course:	Database Management System (DBMS)
Course Code:	CS2144
Semester:	4th — Spring 2026
Section:	BSCS 4-1
Instructor:	Mr. Ihtisham Ullah
University:	Riphah International University
Campus:	Islamabad, I-14

Spring 2026

Table of Contents

1. Introduction 3
2. Objectives 3
3. System Overview 4
4. Entity-Relationship Diagram (ERD) 4
5. Schema Design 5
5.1 Tables & Columns 5
5.2 Constraints 8
5.3 Normalization (up to 3NF) 9
5.4 Relationships 10
6. Views 11
7. SQL Queries 12
7.1 Join Queries 12
7.2 Subqueries 13
7.3 Aggregation Functions 13
8. Triggers 14
9. Sample Data & Outputs 15
10. GitHub Repository 16
11. Conclusion 16

1. Introduction

The Online Bidding Management System is a fully relational database application designed and implemented using Oracle SQL. The system simulates a real-world online auction platform — similar to eBay — where registered users can list products for auction, place competitive bids, and receive automated winner determination.

This project demonstrates the practical application of database concepts studied throughout the semester, including entity-relationship modeling, schema normalization, referential integrity constraints, trigger-based business logic, and complex SQL queries.

The system manages six core entities: Users, Categories, Products, Auctions, Bids, and Auction Winners. Each entity is linked through carefully designed foreign key relationships that enforce data consistency and prevent invalid operations. Advanced features such as PL/SQL triggers ensure that bids are validated in real-time and winners are automatically determined when an auction closes.

The database is implemented on Oracle Database Express Edition (XE) 11g using Oracle SQL Developer as the interface. All schemas conform to the Third Normal Form (3NF), eliminating data redundancy and transitive dependencies.

2. Objectives

The primary objectives of this project are:

- Design a normalized relational database schema conforming to Third Normal Form (3NF).
- Model real-world entities and their relationships using a proper Entity-Relationship Diagram (ERD).
- Implement all tables with appropriate primary keys, foreign keys, CHECK constraints, and UNIQUE constraints in Oracle SQL.
- Enforce business rules through PL/SQL triggers — including bid validation (bid must exceed current highest bid, bid must fall within auction time window) and automatic winner assignment upon auction closure.
- Demonstrate advanced SQL capabilities: multi-table JOIN queries, correlated subqueries, and aggregation functions with GROUP BY and HAVING clauses.
- Create meaningful database views for management reporting — including an auction summary view and a seller performance report.
- Provide a live demonstration of the complete system workflow: user registration, product listing, auction creation, bid placement, and winner determination.

Together, these objectives satisfy the project requirements of demonstrating a Complex Computing Problem (CCP) as defined by the course instructor.

3. System Overview

The Online Bidding Management System supports the following functional modules:

User Management

Users register with a unique email address and are assigned a role of either buyer or seller. The system enforces unique emails and validates user types through CHECK constraints.

Product & Category Management

Sellers list products under specific categories. Each product has a base price, title, and description. Categories are stored in a separate table to achieve normalization.

Auction Management

Each product is associated with exactly one auction. Auctions have a defined start time and end time. A CHECK constraint enforces that end_time > start_time. Auction status transitions from 'active' to 'closed'.

Bid Management

Buyers place bids on active auctions. A PL/SQL trigger validates every bid to ensure: (1) the bid amount exceeds the current highest bid, and (2) the bid timestamp falls within the auction window.

Winner Determination

When an auction is closed (status updated to 'closed'), a PL/SQL trigger automatically selects the buyer with the highest bid and inserts a record into the AUCTION_WINNER table. The UNIQUE constraint on auction_id ensures only one winner per auction.

4. Entity-Relationship Diagram (ERD)

The ERD below illustrates the six entities of the system, their attributes, and the relationships between them.

ERD Description & Notation

The following table describes each entity, its primary key, and its relationships:

Entity	Primary Key	Type	Relationships
USERS	user_id	Strong Entity	Places many BIDS; owns many PRODUCTs; can be AUCTION_WINNER
CATEGORY	category_id	Strong Entity	Contains many PRODUCTs (1:M)
PRODUCT	product_id	Strong Entity	Belongs to one CATEGORY; owned by one USER; has one AUCTION
AUCTION	auction_id	Strong Entity	Linked to one PRODUCT (1:1); has many BIDs; has one WINNER
BID	bid_id	Weak-ish Entity	Placed by one USER on one AUCTION (M:1 each)
AUCTION_WINNER	winner_id	Derived Entity	References one AUCTION and one USER (both 1:1)

Cardinality of Relationships

Relationship	Cardinality	Explanation
USER → PRODUCT	1 : Many	One seller can list many products
CATEGORY → PRODUCT	1 : Many	One category contains many products
PRODUCT → AUCTION	1 : 1	Each product has exactly one auction
AUCTION → BID	1 : Many	One auction receives many bids
USER → BID	1 : Many	One buyer can place many bids
AUCTION → AUCTION_WINNER	1 : 1	Each auction produces exactly one winner
USER → AUCTION_WINNER	1 : Many	A user can win multiple auctions over time

5. Schema Design

5.1 Tables & Columns

The database consists of six tables. Each table is described below with its columns, data types, and constraints.

Table 1: USERS

Column	Data Type	Key/Constraint	Description
user_id	NUMBER	PRIMARY KEY	Unique identifier for each user
name	VARCHAR2(100)	NOT NULL	Full name of the user
email	VARCHAR2(100)	UNIQUE, NOT NULL	Email address — must be unique
password	VARCHAR2(100)	NOT NULL	Stored password for authentication
user_type	VARCHAR2(10)	CHECK ('buyer'/'seller')	Role of user in the system
status	VARCHAR2(10)		Account status (active/inactive)
created_at	DATE	DEFAULT SYSDATE	Account creation timestamp

Table 2: CATEGORY

Column	Data Type	Key/Constraint	Description
category_id	NUMBER	PRIMARY KEY	Unique identifier for category
category_name	VARCHAR2(100)	NOT NULL	Name of the product category

Table 3: PRODUCT

Column	Data Type	Key/Constraint	Description
product_id	NUMBER	PRIMARY KEY	Unique identifier for product
title	VARCHAR2(100)	NOT NULL	Product name/title
description	VARCHAR2(255)		Detailed description of product
base_price	NUMBER	CHECK (> 0)	Minimum starting price for auction
user_id	NUMBER	FOREIGN KEY → USERS	Seller who listed the product
category_id	NUMBER	FOREIGN KEY → CATEGORY	Category the product belongs to

Table 4: AUCTION

Column	Data Type	Key/Constraint	Description
auction_id	NUMBER	PRIMARY KEY	Unique identifier for auction
product_id	NUMBER	FK → PRODUCT, UNIQUE	Product being auctioned (1:1)
start_time	DATE	NOT NULL	When bidding opens
end_time	DATE	CHECK (> start_time)	When bidding closes
auction_status	VARCHAR2(10)		Current status: active / closed

Table 5: BID

Column	Data Type	Key/Constraint	Description
bid_id	NUMBER	PRIMARY KEY	Unique identifier for bid
auction_id	NUMBER	FOREIGN KEY → AUCTION	Auction this bid belongs to
user_id	NUMBER	FOREIGN KEY → USERS	Buyer who placed the bid
bid_amount	NUMBER	CHECK (> 0)	Amount offered — validated by trigger
bid_time	DATE	NOT NULL	Timestamp of bid — validated by trigger

Table 6: AUCTION_WINNER

Column	Data Type	Key/Constraint	Description
winner_id	NUMBER	PRIMARY KEY	Unique identifier for winner record
auction_id	NUMBER	FK → AUCTION, UNIQUE	The auction that was won (1:1)
user_id	NUMBER	FOREIGN KEY → USERS	The winning buyer
winning_bid	NUMBER	NOT NULL	The highest bid amount

ERD diagram:

5.2 Constraints

The following table summarizes all constraints enforced across the schema:

Table	Constraint Type	Column(s)	Rule Enforced
USERS	PRIMARY KEY	user_id	Unique identification of each user
USERS	UNIQUE	email	No two users can share an email
USERS	NOT NULL	email, password, name	Required fields cannot be empty
USERS	CHECK	user_type	Must be 'buyer' or 'seller' only
CATEGORY	PRIMARY KEY	category_id	Unique category identification
PRODUCT	PRIMARY KEY	product_id	Unique product identification
PRODUCT	FOREIGN KEY	user_id → USERS	Seller must exist in USERS table
PRODUCT	FOREIGN KEY	category_id → CATEGORY	Category must exist
PRODUCT	CHECK	base_price	Base price must be greater than 0
AUCTION	PRIMARY KEY	auction_id	Unique auction identification
AUCTION	FOREIGN KEY	product_id → PRODUCT	Product must exist
AUCTION	UNIQUE	product_id	One auction per product only
AUCTION	CHECK	end_time > start_time	End time must follow start time
BID	PRIMARY KEY	bid_id	Unique bid identification
BID	FOREIGN KEY	auction_id → AUCTION	Auction must exist
BID	FOREIGN KEY	user_id → USERS	Bidder must be a registered user
BID	CHECK	bid_amount	Bid amount must be greater than 0
BID	TRIGGER	bid_amount, bid_time	Bid > max bid; bid within time window
AUCTION_WINNER	PRIMARY KEY	winner_id	Unique winner record
AUCTION_WINNER	FOREIGN KEY	auction_id → AUCTION	Auction must exist
AUCTION_WINNER	FOREIGN KEY	user_id → USERS	Winner must be a registered user
AUCTION_WINNER	UNIQUE	auction_id	Only one winner per auction

5.3 Normalization (up to 3NF)

All tables in this database have been normalized up to the Third Normal Form (3NF). Below is the justification for each normal form:

First Normal Form (1NF)

A table is in 1NF if all attributes contain atomic (indivisible) values and there are no repeating groups.

- All columns in every table store a single, atomic value (e.g., name is one field, not first + last combined).
- No column contains arrays, lists, or multi-valued data.
- Each row is uniquely identified by a Primary Key.
- Example: Category is not stored as a comma-separated string inside PRODUCT — it is a separate row in the CATEGORY table, referenced by a foreign key.

Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire Primary Key (no partial dependencies). This applies to tables with composite keys.

- All tables in this system use single-column surrogate primary keys (user_id, product_id, etc.).
- Since there are no composite primary keys, partial dependencies cannot exist by definition.
- All non-key attributes are fully determined by their respective primary key.
- Example: In PRODUCT, all columns (title, description, base_price, user_id, category_id) are determined solely by product_id.

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and there are no transitive dependencies — no non-key attribute is dependent on another non-key attribute.

- Category data: Instead of storing category_name directly in PRODUCT (which would create a transitive dependency product_id → category_id → category_name), we store only category_id as a foreign key. The CATEGORY table holds category_name independently.
- User information: Seller details are not repeated inside PRODUCT. Only user_id is stored, referencing the USERS table.
- Auction data: No product information (title, price) is duplicated in AUCTION or BID. All such data is accessed via foreign keys.
- Winner data: The AUCTION_WINNER table stores only the winning bid amount and references — it does not repeat any bid or user information.

Conclusion: The schema is fully normalized to 3NF with no repeating groups, no partial dependencies, and no transitive dependencies.

5.4 Relationships

The following summarizes all relationships between entities in the schema:

Parent Table	Cardinality	Child Table	Business Rule
USERS	1 : M	PRODUCT	A seller can list many products
CATEGORY	1 : M	PRODUCT	A category can contain many products
PRODUCT	1 : 1	AUCTION	Each product has exactly one auction
AUCTION	1 : M	BID	An auction can receive many bids
USERS	1 : M	BID	A buyer can place many bids
AUCTION	1 : 1	AUCTION_WINNER	Each auction has only one winner
USERS	1 : M	AUCTION_WINNER	A user can win multiple auctions

6. Views

Two views have been created to provide management-level reporting without exposing raw table structures. Views simplify complex queries into reusable virtual tables.

View 1: auction_summary

Purpose: Provides a real-time summary of each auction showing total number of bids received and the current highest bid amount.

```
CREATE VIEW auction_summary AS
SELECT a.auction_id,
       COUNT(b.bid_id)    AS total_bids,
```

```
        MAX(b.bid_amount)  AS highest_bid
FROM auction a
LEFT JOIN bid b ON a.auction_id = b.auction_id
GROUP BY a.auction_id;
```

Key SQL Concepts Used:

- LEFT JOIN — includes auctions that have received no bids yet (total_bids = 0).
- COUNT() aggregate — counts total bids per auction.
- MAX() aggregate — finds the current highest bid.
- GROUP BY — groups results per auction for per-auction statistics.

View 2: seller_report

Purpose: Provides a seller performance report showing how many products each seller has listed on the platform.

```
CREATE VIEW seller_report AS
SELECT u.name,
       COUNT(p.product_id) AS total_products
FROM users u
JOIN product p ON u.user id = p.user id
GROUP BY u.name;
```

Key SQL Concepts Used:

- INNER JOIN — links users with their products.
- COUNT() aggregate — counts total products listed by each seller.
- GROUP BY — groups results by seller name.

Testing Views:

```
SELECT * FROM auction_summary;
SELECT * FROM seller_report;
```

7. SQL Queries

7.1 Join Queries

Query 1: Full System Join — User, Bid, Auction, Product

Retrieves a complete report showing which buyer bid how much on which product in which auction.

```
SELECT u.name        AS buyer_name,
       p.title       AS product title,
       a.auction_id,
       b.bid_amount,
       b.bid_time
FROM users u
JOIN bid     b ON u.user_id      = b.user_id
JOIN auction a ON b.auction id   = a.auction id
JOIN product p ON a.product_id   = p.product_id
ORDER BY b.bid_amount DESC;
```

SQL Features: 4-table INNER JOIN, ORDER BY.

Query 2: Auction Winner Details with Full Join

Retrieves winner name, winning bid amount, and the product they won.

```
SELECT u.name         AS winner_name,
       p.title        AS product,
       aw.winning bid,
       a.auction_id
FROM auction_winner aw
JOIN users   u ON aw.user id    = u.user id
JOIN auction a ON aw.auction_id = a.auction_id
JOIN product p ON a.product_id  = p.product_id;
```

SQL Features: 4-table INNER JOIN spanning all major entities.

7.2 Subqueries

Query 3: Find the Highest Bid Per Auction (Correlated Subquery)

Returns all bid records where the bid_amount equals the maximum bid for that auction — identifying the winning bid row.

```
SELECT *
FROM bid b
WHERE bid amount = (
    SELECT MAX(bid_amount)
    FROM bid
    WHERE auction id = b.auction id
);
```

SQL Features: Correlated subquery — the inner query references the outer query's current row.

Query 4: Find Users Who Have Never Placed a Bid (NOT IN Subquery)

Identifies registered buyers who have not participated in any auction.

```
SELECT name, email
FROM users
WHERE user_id NOT IN (
    SELECT DISTINCT user_id FROM bid
);
```

SQL Features: NOT IN subquery with DISTINCT.

Query 5: Products That Have Active Auctions (EXISTS Subquery)

```
SELECT title, base price
FROM product p
WHERE EXISTS (
    SELECT 1 FROM auction a
    WHERE a.product_id   = p.product_id
    AND   a.auction_status = 'active'
);
```

SQL Features: EXISTS correlated subquery.

7.3 Aggregation Functions

Query 6: Total Bids and Highest Bid Per Auction

```
SELECT auction_id,
       COUNT(*)          AS total_bids,
       MAX(bid_amount)   AS highest_bid,
       MIN(bid_amount)   AS lowest_bid,
       AVG(bid_amount)   AS average_bid
FROM bid
GROUP BY auction_id;
```

SQL Features: COUNT, MAX, MIN, AVG aggregate functions with GROUP BY.

Query 7: Count of Active Auctions

```
SELECT COUNT(*) AS active_auctions
FROM auction
WHERE auction_status = 'active';
```

Query 8: Auctions With More Than 1 Bid (HAVING Clause)

```
SELECT auction_id, COUNT(*) AS bid_count
FROM bid
GROUP BY auction_id
HAVING COUNT(*) > 1;
```

SQL Features: GROUP BY with HAVING filter — restricts aggregate results.

8. Triggers

Triggers are PL/SQL blocks that execute automatically in response to data modification events (INSERT, UPDATE, DELETE). This project uses two triggers to enforce business rules that cannot be implemented with simple SQL constraints.

Trigger 1: validate_bid — Bid Validation (BEFORE INSERT on BID)

This trigger fires before every INSERT on the BID table and validates two rules: (1) the new bid must exceed the current highest bid, and (2) the bid timestamp must fall within the auction's active time window.

```
CREATE OR REPLACE TRIGGER validate_bid
BEFORE INSERT ON bid
FOR EACH ROW
DECLARE
    max_bid       NUMBER;
    auction_start DATE;
    auction_end   DATE;
BEGIN
```

```
    -- Get current highest bid for this auction
    SELECT NVL(MAX(bid_amount), 0)
    INTO max_bid
    FROM bid
    WHERE auction_id = :NEW.auction_id;

    -- Get auction time window
    SELECT start_time, end_time
    INTO auction_start, auction_end
    FROM auction
    WHERE auction_id = :NEW.auction_id;

    -- Rule 1: New bid must be higher than previous highest
    IF :NEW.bid_amount <= max_bid THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Bid must be higher than the current highest bid');
    END IF;

    -- Rule 2: Bid must be placed within auction time window
    IF :NEW.bid_time < auction_start
    OR :NEW.bid_time > auction_end THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Bid must be placed within the auction time window');
    END IF;
END;
/
```

Trigger 2: assign_winner — Auto Winner Assignment (AFTER UPDATE on AUCTION)

This trigger fires after an UPDATE on the AUCTION table. When auction_status is changed to 'closed', the trigger automatically finds the buyer with the highest bid and inserts the winner record into AUCTION_WINNER.

```
CREATE SEQUENCE auction_winner_seq
START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER assign_winner
AFTER UPDATE ON auction
FOR EACH ROW
WHEN (NEW.auction_status = 'closed')
DECLARE
    v_user NUMBER;
    v_bid  NUMBER;
BEGIN
    -- Find the winning user and bid amount
    SELECT user_id, bid_amount
    INTO v_user, v_bid
    FROM bid
```

```
    WHERE auction_id = :NEW.auction_id
    AND   bid_amount = (
        SELECT MAX(bid_amount)
        FROM bid
        WHERE auction_id = :NEW.auction_id
    );

    -- Insert winner record
    INSERT INTO auction winner
    VALUES (auction_winner_seq.NEXTVAL,
            :NEW.auction_id, v_user, v_bid);
END;
/
```

9. Sample Data & Outputs

9.1 Sample Insert Statements

```
-- Insert Users
INSERT INTO users VALUES (1,'Ali','ali@email.com','pass123','seller','active',SYSDATE);
INSERT INTO users VALUES
(2,'Ahmed','ahmed@email.com','pass123','buyer','active',SYSDATE);
INSERT INTO users VALUES (3,'Sara','sara@email.com','pass123','buyer','active',SYSDATE);

-- Insert Category
INSERT INTO category VALUES (1, 'Electronics');
INSERT INTO category VALUES (2, 'Furniture');

-- Insert Products
INSERT INTO product VALUES (1,'Laptop','Gaming Laptop 16GB',50000,1,1);
INSERT INTO product VALUES (2,'Chair','Ergonomic Office Chair',15000,1,2);

-- Insert Auctions
INSERT INTO auction VALUES (1,1,SYSDATE,SYSDATE+1,'active');
INSERT INTO auction VALUES (2,2,SYSDATE,SYSDATE+2,'active');

-- Insert Bids
INSERT INTO bid VALUES (1,1,2,55000,SYSDATE);
INSERT INTO bid VALUES (2,1,3,62000,SYSDATE);
INSERT INTO bid VALUES (3,2,2,17000,SYSDATE);

COMMIT;
```

9.2 Demo Flow

The following sequence demonstrates the complete system workflow for a live demonstration:

#	Action	SQL Command
1	View all users	`SELECT * FROM users;`
2	View all products	`SELECT * FROM product;`
3	View active auctions	`SELECT * FROM auction WHERE auction_status = 'active';`
4	Place valid bid	`INSERT INTO bid VALUES (1, 1, 2, 55000, SYSDATE);`
5	Place invalid bid (lower) — shows trigger error	`INSERT INTO bid VALUES (2, 1, 2, 50000, SYSDATE);`
6	Place higher bid	`INSERT INTO bid VALUES (3, 1, 3, 62000, SYSDATE);`
7	Close the auction	`UPDATE auction SET auction_status = 'closed' WHERE auction_id = 1;`
8	Verify winner auto-assigned	`SELECT * FROM auction_winner;`
9	View auction summary report	`SELECT * FROM auction_summary;`
10	View seller performance	`SELECT * FROM seller_report;`

10. GitHub Repository

The complete project source code, scripts, and documentation are hosted on GitHub. The repository is structured as follows:

```
online-bidding-system/
├── ERD.png                 ← Entity-Relationship Diagram image
├── schema.sql              ← All CREATE TABLE statements
├── triggers.sql            ← PL/SQL trigger definitions
├── views.sql               ← View creation scripts
├── queries.sql             ← All required SQL queries
├── insert.sql              ← Sample data inserts
├── report.pdf              ← Final project report (this document)
└── README.md               ← Project description and setup guide
```

GitHub Repository Link:

[Ubaid-2/Online-Bidding-System-DBMS: Oracle SQL based Online Bidding Management System]

.

11. Conclusion

The Online Bidding Management System successfully demonstrates the practical application of relational database concepts as taught in the Database Management System course. The project goes beyond basic CRUD operations and implements a realistic, constraint-driven, trigger-enabled system.

The following key achievements were accomplished:

- A fully normalized schema conforming to Third Normal Form (3NF) was designed, eliminating data redundancy and transitive dependencies across all six tables.
- Comprehensive integrity constraints — PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK — were enforced to maintain data consistency at the database level.
- Two PL/SQL triggers were implemented: one to validate bid logic in real time (bid must exceed current highest bid and fall within the auction window), and one to automatically assign the winner when an auction closes.
- Advanced SQL queries were written using multi-table JOINs (across four tables), correlated subqueries, EXISTS/NOT IN subqueries, and aggregation functions (COUNT, MAX, MIN, AVG) with GROUP BY and HAVING clauses.
- Two analytical views (auction_summary and seller_report) were created to demonstrate management-level reporting capabilities.
- A complete live demonstration workflow was prepared covering user registration, product listing, auction creation, valid and invalid bid placement, auction closure, and automated winner determination.

This project simulates real-world platforms such as eBay and demonstrates that a well-designed relational database can enforce complex business rules reliably at the database layer — without depending on application-level validation. The experience has strengthened practical understanding of database design, normalization theory, SQL implementation, and PL/SQL programming.

Submitted by:
Muhammad Ubaid ur Rehman
SAP ID: 66784
BSCS 4-1 | Spring 2026